

LOGOS BASELINE RUN NOTES

Fixed-Budget Control Experiment for Language Model Efficiency Research

Field	Details
Project	Logos
Run Name	logos_base_32m_control
Model Family	Decoder-only causal Transformer language model
Dataset	OpenWebText subset
Tokenizer	GPT-2 BPE tokenizer
Purpose	Create a measurable baseline for future efficiency experiments
Prepared For	Logos model optimization research track

Core research aim: make language models more capable per parameter, FLOP, memory byte, latency unit, and watt.

1. Project Aim

The aim of the Logos project is to build and evaluate efficient language models that become more capable while using fewer resources. The focus is not only on training a language model, but on measuring how much capability the model achieves per parameter, per FLOP, per memory byte, per second of latency, and per watt of energy.

In this run, a baseline decoder-only GPT-style language model was trained from scratch to establish a measurable control experiment. This baseline will be used as the reference point for future architecture, data, compression, quantization, and distillation experiments.

2. Baseline Model Type

The model used in this run is a decoder-only causal language model, similar in architecture style to GPT and LLaMA-like models. It learns next-token prediction: given previous tokens, predict the next token. This is called causal language modeling.

Given previous tokens -> predict the next token

Example:

Input: The future of language models

Target: is

Because the model predicts tokens from left to right, it can generate text after training. This makes decoder-only causal language modeling the correct baseline for an efficient generative language model project.

3. Dataset and Tokenizer

Component	Description
Dataset	OpenWebText subset
Number of samples	10,000 text samples
Tokenizer	GPT-2 tokenizer
Objective	Next-token prediction
Train/validation split	90 percent training, 10 percent validation

The GPT-2 tokenizer was used because it is a standard byte-pair encoding tokenizer for English language modeling experiments. This also makes the baseline easier to compare with other GPT-style language modeling setups.

4. Architecture Used

The baseline architecture was a standard Transformer decoder model with causal self-attention.

Component	Value
Architecture	Decoder-only Transformer
Layers	8
Attention heads	8
Embedding dimension	256
Context length	256 tokens
Dropout	0.2
Parameters	32,159,825
Activation	ReLU
Normalization	LayerNorm
Attention type	Causal self-attention

The model uses causal masking, meaning each token can only attend to earlier tokens and cannot access future tokens. This constraint allows the model to learn natural left-to-right generation.

5. Training Configuration

Setting	Value
Batch size	64
Block size	256
Maximum iterations	1500
Learning rate	3e-4
Optimizer	AdamW
Weight decay	0.1
Gradient clipping	1.0
Mixed precision	Enabled
Device	NVIDIA RTX 6000 Ada Generation
Available VRAM	Approximately 48 GB

The run was performed on a high-end GPU system available for a limited time. Therefore, the goal was to create a strong, measurable baseline quickly and record scientific metrics before moving to later experiments.

6. Final Baseline Results

Metric	Value
Run name	logos_base_32m_control
Parameters	32,159,825
Best iteration	1500
Best validation loss	5.7716
Final validation loss	5.7634
Final validation perplexity	318.42
Checkpoint size	122.71 MB
Peak VRAM	10.84 GB
Training time	156.42 seconds, approximately 2.61 minutes
Training speed	Approximately 158k tokens per second
Generation speed	98.04 tokens per second
Generation latency	10.20 ms per token
Average GPU power	250.06 W
Estimated GPU energy	10.78 Wh

7. Training Behavior

The validation loss steadily decreased during training, as shown in the approximate trend below.

Iteration	Approximate Validation Loss
0	10.86
150	6.99
300	6.46
600	6.08
900	5.88
1200	5.79
1500	5.77

This shows that the model was still learning at the end of the run. The validation loss had not fully flattened, so this experiment should be treated as a fixed-budget baseline rather than a fully converged training run.

8. Interpretation

This baseline successfully proves that the Logos training pipeline works end to end. The run confirms that the system can:

- Load OpenWebText data.
- Tokenize text using GPT-2 BPE.
- Train a GPT-style decoder-only Transformer.
- Log training and validation loss.
- Measure perplexity.
- Track VRAM usage.
- Measure training throughput.
- Measure generation latency.
- Save checkpoint and result files.

The final perplexity of approximately 318 is expected for a small model trained for a short time on a limited dataset subset. The main success of this run is not high language quality yet, but the creation of a measurable baseline for future optimization experiments.

9. Why This Run Matters

This run gives the project a control point. Future experiments can now be compared against the baseline using validation loss, perplexity, parameter count, training speed, VRAM usage, checkpoint size, generation latency, and energy consumption.

This is important because the goal of Logos is not just to improve model performance, but to improve performance per resource. A future model is only meaningfully better if it improves the capability-to-resource tradeoff.

10. Scientific Baseline Statement

This run establishes a controlled baseline for the Logos project: a 32.16M-parameter decoder-only GPT-style Transformer trained on a 10,000-sample OpenWebText subset using GPT-2 tokenization. The model achieved a final validation loss of 5.7634 and a validation perplexity of 318.42 after 1500 iterations, with a peak VRAM usage of 10.84 GB and generation latency of 10.20 ms/token. Since validation loss was still decreasing at the end of training, this run is considered a fixed-budget control baseline for future efficiency experiments.

11. Limitations of This Run

Limitation	Explanation
Small dataset subset	Only 10,000 OpenWebText samples were used.
Short training time	Only 1500 iterations were run.
Not fully converged	Validation loss was still decreasing.
Basic architecture	The baseline used a standard Transformer block with ReLU and LayerNorm.
No architecture optimization yet	No RMSNorm, SwiGLU, weight tying, grouped-query attention, or other optimizations were tested.
No distillation	The model was trained only from raw next-token labels.
No quantization	The model was evaluated in normal precision without compression.

Therefore, this run is not the final optimized Logos model. It is the first measurable reference point for a scientific optimization process.

12. Next Experiment

The best next experiment should change only one variable. The recommended next experiment is weight tying.

Experiment	Hypothesis	Expected Benefit
Weight tying	Sharing the input token embedding and output language-model head weights will reduce parameter count and checkpoint size while maintaining similar or slightly better validation loss.	Better parameter efficiency, smaller checkpoint size, possible generalization improvement, and minimal code change.

For a fair comparison, all other settings should remain unchanged:

- Same dataset.
- Same tokenizer.
- Same batch size.
- Same block size.
- Same model depth.
- Same embedding size.
- Same learning rate.
- Same number of iterations.
- Same seed.

13. One-Line Summary

The first Logos baseline successfully trained a 32.16M-parameter decoder-only GPT-style language model from scratch and established a measurable control point for future experiments focused on making language models more powerful, faster, smaller, and more efficient.